

Relaxed Multi-Trip Truck–Drone–Robot MILP with En-Route Charging

Formulation Following VRP-DR (Malik et al., arXiv:2505.23584),
with the Relaxed Penalized Objective of the Original TDRP Formulation

This document writes out the mixed-integer linear program (MILP) implemented in `model/capped_flexible_docking_ordered_sortie_model.py`, restructured to follow the mathematical format of the VRP-DR paper (Malik et al., “Collaborative Last-Mile Delivery: A Multi-Platform Vehicle Routing Problem With En-route Charging,” arXiv:2505.23584) and the relaxed penalized objective of the original truck–drone–robot formulation (Eq. 12 of the TDRP paper). Every symbol, set, and constraint corresponds to named variables, constraints, or helper blocks in the code (items marked *new* are specified here and land in the code with the phased migration described below), so the formulation can be checked against the source directly.

Implementation status. The migration from the previous strict single-trip model proceeds in phases: (1) cyclic-sortie timing fix (`dep_t` ready times); (2) relaxed objective with excess variables, Γ makespan, and α blend; (3) energy-per-sortie filter; (4) trip-indexed multi-trip; (5) full battery/charging variables. Until a phase lands, the corresponding *new* items are specification, not code.

1 Sets and Indices

Symbol	Meaning
$C = \{1, \dots, n\}$	Customers. $n = C $.
$P = \{0, n + 1\}$	Depots: 0 is the start depot, $n + 1$ is the end depot.
V	$V = \{0, 1, \dots, n + 1\} = C \cup P$: all nodes.
$A \subseteq V \times V$	Directed arc set (from <code>arcs.csv</code>); $(i, j) \in A$ means a truck may travel directly from i to j .
$A^+ = A \cup \{(i, i) : i \in V\}$	Leg set for sortie enumeration, extended with self-loops for <i>cyclic</i> sorties (Section 3).
$T = \{1, \dots, T \}$	Trucks, $ T = \text{NUM_TRUCKS}$.
$D = \{1, \dots, D \}$	Drones, after lossless fleet trimming (Section 8).
$R = \{1, \dots, R \}$	Robots, after lossless fleet trimming.
$Q = \{1, \dots, Q_{\max}\}$	Trip indices: each physical drone/robot may fly up to $Q_{\max}$ sorties (multi-trip).
S^d	Set of candidate ordered drone sorties (Section 3).
S^r	Set of candidate ordered robot sorties.

2 VRP-DR Feature Map

The five distinguishing features of VRP-DR are adopted as follows ($\checkmark$ = formulated in this document):

VRP-DR feature	Where it lives in this formulation
Multi-visit service	Ordered sorties serve up to <code>MAX_CUSTOMERS_PER_SORTIE</code> customers (Section 3) $\checkmark$
Multi-trip operations	Trip-indexed platforms (d, q) with inter-trip sequencing (Sections 5, 7.6) $\checkmark$
Flexible docking	Independent launch/recovery truck assignment $h^L \neq h^R$ (Section 7.5) $\checkmark$
Cyclic and acyclic operations	Self-loop legs $(i, i) \in A^+$ plus truck ready times dep^t (Sections 3, 7.8) $\checkmark$

In addition, the objective carries the VRP-DR cost–makespan blend $\alpha \cdot \text{OpCost} + (1 - \alpha) \Gamma$ (Section 6), and the hard capacity/endurance constraints of the previous model are relaxed into the penalized form of the original TDRP formulation’s Eq. 12 (Sections 7.2, 7.4).

3 Ordered Sortie Candidates

Before the MILP is built, `generate_ordered_sorties` enumerates, for every leg $(i, k) \in A^+$ and every ordered subsequence of customers of length $1, \dots, \text{MAX_CUSTOMERS_PER_SORTIE}$, a candidate *ordered sortie* s with:

- launch node $\iota(s) = i$, recovery node $\kappa(s) = k$ (with $i = k$ allowed: a *cyclic* sortie);
- ordered customer sequence $\sigma(s) = (\sigma_1(s), \dots, \sigma_{m(s)}(s))$, $\sigma_l(s) \in C$;
- payload $q(s) = \sum_{c \in \sigma(s)} q_c$, kept only if $q(s) \leq$ platform capacity (hard, faithful to Eq. 3 of the original formulation);
- travel distance $\delta(s) = \sum_{(u,v) \in \text{edges}(s)} \text{dist}_{uv}$ (drone: `distance_matrix`; robot: `robot_distance_matrix`);
- duration $\tau(s) = \sum_{(u,v) \in \text{edges}(s)} \text{time}_{uv}$;
- cost $\gamma(s) = \sum_{(u,v) \in \text{edges}(s)} \text{cost}_{uv}$;
- **energy** $e(s) = (a_p + b_p q(s)) \delta(s)$ in Wh, with platform energy coefficients (a_p, b_p) (Section 4). Because the sequence and per-leg payloads are known at enumeration time, energy is precomputed exactly as VRP-DR’s payload-dependent drone formula requires, but no energy nonlinearity ever enters the MILP.

where

$$\text{edges}(s) = ((\iota(s), \sigma_1(s)), (\sigma_1(s), \sigma_2(s)), \dots, (\sigma_{m(s)}(s), \kappa(s))).$$

For a cyclic leg (i, i) , $\text{edges}(s)$ is the genuine cycle $i \rightarrow \sigma_1 \rightarrow \dots \rightarrow \sigma_m \rightarrow i$. For each leg, only the `TOP_SORTIES_PER_TRUCK_LEG` candidates with the most customers (ties broken by lowest cost, then duration) are kept. S^d uses drone time/distance matrices and (Q_d, B_d) ; S^r uses robot time/distance matrices and (Q_r, B_r) .

Generation filters (relaxed model). Because endurance violations are now *priced* rather than forbidden, generation keeps sorties with $e(s) \leq E_{\text{slack}} \cdot B_p$ ($E_{\text{slack}} \geq 1$, default 1.5), so the λ_E penalties have candidates to price. The legacy distance caps E_d, E_r remain as documented non-binding sanity caps. Sorties with $e(s) > B_p$ are still discarded only when $E_{\text{slack}} = 1$.

Therefore, an “integer optimal solution” from the solver is optimal over the retained capped candidate-sortie sets S^d and S^r . It is not a proof of global optimality over every possible ordered sortie in the unrestricted problem.

4 Parameters

Symbol	Meaning (source)
q_i	Demand of customer $i \in C$ (<code>customers.csv</code>).
$e_i, \bar{e}_i$	Time-window open/close time of customer i (<code>open_time</code> , <code>close_time</code>).
θ_i	Service time of customer i (<code>service_time</code>).
dist_{ij}	Truck/drone travel distance, $i, j \in V$ (<code>distance_matrix</code>).
tt_{ij}	Truck travel time, $i, j \in V$ (<code>truck_time_matrix</code>).
tt_{ij}^p	Truck travel time departing i at the start of period p , $i, j \in V$ (<code>truck_time_matrix_{profile}_p{p}.csv</code> ; Section 9).
dt_{ij}	Drone travel time, $i, j \in V$ (<code>drone_time_matrix</code>).
rdist_{ij}	Robot travel distance, $i, j \in V$ (<code>robot_distance_matrix</code>).
rt_{ij}	Robot travel time, $i, j \in V$ (<code>robot_time_matrix</code>).

Q_t, Q_d, Q_r	Truck / drone / robot payload capacity.
B_d, B_r	Usable battery capacity, drone / robot (Wh). Defaults: $B_d = 2000$ (derived: reproduces the 40 km Sacramento range at 45 Wh/km and 90% DoD), $B_r = 1260$ (Starship Gen 3 spec).
a_d, b_d	Drone energy coefficients (Wh/km): $e = (a_d + b_d q) \delta$; defaults (30, 3), i.e. 45 Wh/km at 5 kg.
a_r, b_r	Robot energy coefficients (Wh/km); defaults (12, 0.6), i.e. 18 Wh/km at 10 kg.
r_d^c, r_r^c	En-route charging rates, drone / robot (W). Defaults 1000 / 400 (truck-side electrical limits).
$Q_{\max}$	Maximum trips per physical drone/robot (<i>new</i> ; suggest 2–3).
τ_{turn}	Minimum turnaround time between consecutive trips of one platform (<i>new</i> ; 0 if charging models it).
E_{slack}	Endurance slack at sortie generation, ≥ 1 (<i>new</i> ; default 1.5).
$T_{\max}$	Route-duration limit (8 h in the real-world DeKalb data).
C_w, C_{veh}	Truck time-cost and distance-cost coefficients.
C_w^d, C_{drone}^d	Drone time-cost and distance-cost coefficients.
C_w^r, C_{rob}^r	Robot time-cost and distance-cost coefficients.
f_t, f_d, f_r	Fixed activation cost per used truck / selected drone sortie / selected robot sortie (values 30, 10, 8 from Malik et al., VRP-DR, Table 3).
$\alpha \in [0, 1]$	Weight on operating cost vs. makespan in the objective (<i>new</i> ; VRP-DR uses 0.5).
λ_Q	Penalty per kg of truck capacity excess (<i>new</i> ; suggest 10–20).
λ_E^d, λ_E^r	Penalty per Wh of drone/robot battery excess (<i>new</i> ; suggest 5–10 per km-equivalent).
λ_T, λ_W	Penalty weights for route-duration excess and time-window lateness.
μ_d^{dep}	Drones a used truck may launch from the depot.
μ_d^{trk}	Distinct drones a single truck may use.
μ_r^{dep}	Robots a used truck may launch from the depot.
μ_r^{trk}	Distinct robots a single truck may use.
M	Big-M, $M = \max(T_{\max}, 10 \cdot V \cdot \max(tt, dt, rt), 10000)$.

Penalty scaling. Each λ must exceed the maximum marginal operating-cost saving from one unit of violation (exact-penalty theory) but not by orders of magnitude: with $\lambda_T = 1000$ against a $\sim \$68$ solution, 0.07 units of violation swamp the entire objective (observed: route-duration penalty reached $\sim 97\%$ of the objective before the per-instance $T_{\max}$ repair). Suggested starting values are in the table; calibrate per instance family.

4.1 Travel-cost coefficients

$$\text{tcost}_{ij} = \text{tt}_{ij} C_w + \text{dist}_{ij} C_{veh}, \quad (i, j) \in A \quad (1)$$

$$\text{dcost}_{ij} = \text{dt}_{ij} C_w^d + \text{dist}_{ij} C_{drone}^d, \quad i, j \in V \quad (2)$$

$$\text{rcost}_{ij} = \text{rt}_{ij} C_w^r + \text{rdist}_{ij} C_{rob}^r, \quad i, j \in V \quad (3)$$

Sortie cost $\gamma(s)$ in Section 3 sums dcost (drone sorties, $s \in S^d$) or rcost (robot sorties, $s \in S^r$) over edges(s).

5 Decision Variables

Symbol	Code name	Meaning
$x_{v,i,j} \in \{0, 1\}$	<code>x[v,i,j]</code>	Truck v traverses arc $(i, j) \in A$.
$\eta_v \in \{0, 1\}$	<code>used_truck[v]</code>	Truck v is used.
$\text{vis}_{v,i} \in \{0, 1\}$	<code>visit[v,i]</code>	Truck v visits customer $i \in C$.
$u_{v,i} \in [0, n]$	<code>u[v,i]</code>	MTZ rank of customer i on truck v 's route.
$y_{d,q,s} \in \{0, 1\}$	<i>new</i>	Drone d flies ordered sortie $s \in S^d$ as its q -th trip.
$\eta_d^d \in \{0, 1\}$	<i>new</i>	Drone d is used (at least one trip).
$h_{v,d,q,s}^L \in \{0, 1\}$	<i>new</i>	Truck v launches drone d 's q -th trip (sortie s).
$h_{v,d,q,s}^R \in \{0, 1\}$	<i>new</i>	Truck v recovers drone d 's q -th trip (sortie s).
$\xi_{v,d} \in \{0, 1\}$	<code>drone_truck[v,d]</code>	Drone d interacts with truck v (launch and/or recovery, any trip).

$z_{r,q,s} \in \{0,1\}$	<i>new</i>	Robot r performs ordered sortie $s \in S^r$ as its q -th trip.
$\eta_r^r \in \{0,1\}$	<i>new</i>	Robot r is used (at least one trip).
$g_{v,r,q,s}^L \in \{0,1\}$	<i>new</i>	Truck v launches robot r 's q -th trip.
$g_{v,r,q,s}^R \in \{0,1\}$	<i>new</i>	Truck v recovers robot r 's q -th trip.
$\zeta_{v,r} \in \{0,1\}$	<code>robot_truck[v,r]</code>	Robot r interacts with truck v (any trip).
$lt_{d,q}^d \geq 0$	<i>new</i>	Launch time of drone d 's q -th trip.
$rt_{d,q}^d \geq 0$	<i>new</i>	Recovery (finish) time of drone d 's q -th trip.
$lt_{r,q}^r \geq 0$	<i>new</i>	Launch time of robot r 's q -th trip.
$rt_{r,q}^r \geq 0$	<i>new</i>	Recovery time of robot r 's q -th trip.
$b_{d,q}^d \geq 0$	<i>new</i>	Drone d battery at the start of trip q (Wh).
$c_{d,q}^d \geq 0$	<i>new</i>	Energy charged to drone d between trips q and $q+1$ (Wh).
$b_{r,q}^r \geq 0$	<i>new</i>	Robot r battery at the start of trip q (Wh).
$c_{r,q}^r \geq 0$	<i>new</i>	Energy charged to robot r between trips q and $q+1$ (Wh).
$a_{v,i}^t \geq 0$	<code>a_t[v,i]</code>	Arrival time of truck v at node $i \in V$.
$dep_{v,i}^t \geq 0$	<i>new</i>	Ready (departure) time of truck v at node $i \in V$; the truck may wait for returning platforms.
$a_i^d \geq 0$	<code>a_d[i]</code>	Drone arrival time at node $i \in V$.
$a_i^r \geq 0$	<code>a_r[i]</code>	Robot arrival time at node $i \in V$.
$\ell_i^t \geq 0$	<code>late_t[i]</code>	Truck time-window lateness at customer $i \in C$.
$\ell_i^d \geq 0$	<code>late_d[i]</code>	Drone time-window lateness at customer $i \in C$.
$\ell_i^r \geq 0$	<code>late_r[i]</code>	Robot time-window lateness at customer $i \in C$.
$\varepsilon_v^Q \geq 0$	<i>new</i>	Truck v capacity excess (kg).
$\varepsilon_{d,q}^{Ed} \geq 0$	<i>new</i>	Drone d battery excess on trip q (Wh).
$\varepsilon_{r,q}^{Er} \geq 0$	<i>new</i>	Robot r battery excess on trip q (Wh).
$\rho_v^t \geq 0$	<code>route_late_t[v]</code>	Route-duration excess of truck v over $T_{\max}$.
$\rho^* \geq 0$	<code>route_late_global</code>	Global route-duration excess.
$\Gamma \geq 0$	<i>new</i>	Makespan: latest completion time over all trucks and platform trips.

5.1 Derived service and trip-use indicators

$$\sigma_i^t = \sum_{v \in T} \text{vis}_{v,i}, \quad i \in C \quad (4)$$

$$\sigma_j^d = \sum_{d \in D} \sum_{q \in Q} \sum_{\substack{s \in S^d \\ j \in \sigma(s)}} y_{d,q,s}, \quad j \in C \quad (5)$$

$$\sigma_j^r = \sum_{r \in R} \sum_{q \in Q} \sum_{\substack{s \in S^r \\ j \in \sigma(s)}} z_{r,q,s}, \quad j \in C \quad (6)$$

$$\text{used}_{d,q}^d = \sum_{s \in S^d} y_{d,q,s} \leq 1, \quad d \in D, q \in Q \quad (7)$$

$$\text{used}_{r,q}^r = \sum_{s \in S^r} z_{r,q,s} \leq 1, \quad r \in R, q \in Q \quad (8)$$

$\text{used}_{d,q}^d$ ($\text{used}_{r,q}^r$) is a linear expression, not a new variable: it is 1 iff drone d (robot r) flies a q -th trip.

6 Objective Function

6.1 Variable (distance/time) cost

$$\text{VarCost}^t = \sum_{v \in T} \sum_{(i,j) \in A} \text{tcost}_{ij} x_{v,i,j} \quad (9)$$

$$\text{VarCost}^d = \sum_{d \in D} \sum_{q \in Q} \sum_{s \in S^d} \gamma(s) y_{d,q,s} \quad (10)$$

$$\text{VarCost}^r = \sum_{r \in R} \sum_{q \in Q} \sum_{s \in S^r} \gamma(s) z_{r,q,s} \quad (11)$$

$$\text{VarCost} = \text{VarCost}^t + \text{VarCost}^d + \text{VarCost}^r \quad (12)$$

6.2 Fixed activation cost

$$\text{FixCost}^t = f_t \sum_{v \in T} \eta_v \quad (13)$$

$$\text{FixCost}^d = f_d \sum_{d \in D} \eta_d^d \quad (14)$$

$$\text{FixCost}^r = f_r \sum_{r \in R} \eta_r^r \quad (15)$$

$$\text{FixCost} = \text{FixCost}^t + \text{FixCost}^d + \text{FixCost}^r \quad (16)$$

where $\eta_d^d \geq y_{d,q,s}$ for all q, s and $\eta_d^d \leq \sum_{q \in Q} \sum_{s \in S^d} y_{d,q,s}$ (robot analogues for η_r^r), so $\eta_d^d = 1$ iff drone d flies at least one trip.

Fixed platform cost is per *used platform* (Malik et al., VRP-DR, Table 3): a platform flying several trips still activates once. With $Q_{\max} = 1$ this coincides with the legacy per-sortie charge.

6.3 Operating cost, makespan blend, and penalties

$$\text{OpCost} = \text{VarCost} + \text{FixCost} \quad (17)$$

$$\text{Pen}_Q = \lambda_Q \sum_{v \in T} \varepsilon_v^Q \quad (18)$$

$$\text{Pen}_E = \lambda_E^d \sum_{d,q} \varepsilon_{d,q}^{Ed} + \lambda_E^r \sum_{r,q} \varepsilon_{r,q}^{Er} \quad (19)$$

$$\text{Pen}_T = \lambda_T \rho^* \quad (20)$$

$$\text{Pen}_W = \lambda_W \left(\sum_{i \in C} \ell_i^t + \sum_{i \in C} \ell_i^d + \sum_{i \in C} \ell_i^r \right) \quad (21)$$

$$\text{TotalPen} = \text{Pen}_Q + \text{Pen}_E + \text{Pen}_T + \text{Pen}_W \quad (22)$$

6.4 Objective

$$\min \quad P = \alpha \cdot \text{OpCost} + (1 - \alpha) \Gamma + \text{TotalPen} \quad (23)$$

This is the original TDRP formulation's penalized objective (Eq. 12: operating cost plus $\lambda_Q, \lambda_E^d, \lambda_E^r, \lambda_T, \lambda_W$ violation prices) with the VRP-DR cost-makespan scalarization $\alpha \cdot \text{OpCost} + (1 - \alpha) \Gamma$ in place of pure operating cost. Penalties sit *outside* the α blend: they are already in cost units, so feasibility pricing must not vanish as $\alpha \rightarrow 0$. With $\alpha = 1$, Γ dropped, and $\lambda_Q, \lambda_E^d, \lambda_E^r \rightarrow \infty$, (23) reproduces the previous strict model; the code exposes this via a `relaxed_objective` flag so strict-optimal and relaxed-optimal results remain distinguishable in provenance.

7 Constraints

7.1 Customer service (assignment)

$$\sigma_i^t + \sigma_i^d + \sigma_i^r = 1, \quad i \in C \quad (24)$$

Every customer is served exactly once, by exactly one of truck, drone, or robot. Hard in both friend formulations.

7.2 Relaxed truck capacity

$$\varepsilon_v^Q \geq \sum_{i \in C} q_i \text{vis}_{v,i} - Q_t, \quad v \in T \quad (25)$$

$$\varepsilon_v^Q \geq 0, \quad v \in T \quad (26)$$

The hard truck-capacity constraint is replaced by the excess variable ε_v^Q priced at λ_Q in (18) — the λ_Q term of the original formulation’s Eq. 12.

7.3 Drone/robot per-sortie payload (hard)

$$q(s) y_{d,q,s} \leq Q_d, \quad d \in D, q \in Q, s \in S^d \quad (27)$$

$$q(s) z_{r,q,s} \leq Q_r, \quad r \in R, q \in Q, s \in S^r \quad (28)$$

Stays hard: the original formulation’s Eq. 3 states these as hard conditionals ($q_j \leq Q_d$ if served by drone), not penalized terms, and generation already enforces them. Relaxing them buys nothing.

7.4 Relaxed energy/endurance

For every $d \in D, q \in Q$ (robot analogues for $r \in R$):

$$\varepsilon_{d,q}^{Ed} \geq \sum_{s \in S^d} e(s) y_{d,q,s} - b_{d,q}^d, \quad (29)$$

$$\varepsilon_{d,q}^{Ed} \geq 0 \quad (30)$$

The hard distance-endurance constraints are replaced by per-trip battery-excess variables priced at λ_E^d, λ_E^r in (19) — the λ_E^d, λ_E^r terms of Eq. 12, now in energy units and charged against the trip’s battery level $b_{d,q}^d$ (Section 7.7), not the nameplate capacity. Energy binds for the drone (B_d sized to the 40 km range); for the robot the battery is $\sim 5\times$ oversized for 12 km sorties, so ε^{Er} is structurally near-zero at our scale. With $Q_{\max} = 1$, $b_{d,1}^d = B_d$ and (29) collapses to the single-trip priced excess over nameplate capacity.

7.5 Drone sortie–truck assignment and synchronization

For every $d \in D, q \in Q, s \in S^d$:

$$\sum_{v \in T} h_{v,d,q,s}^L = y_{d,q,s} \quad (31)$$

$$\sum_{v \in T} h_{v,d,q,s}^R = y_{d,q,s} \quad (32)$$

and for every $v \in T$:

$$h_{v,d,q,s}^L \leq \begin{cases} \eta_v, & \iota(s) = 0 \\ \text{vis}_{v,\iota(s)}, & \iota(s) \in C \\ 0, & \text{otherwise} \end{cases} \quad (33)$$

$$h_{v,d,q,s}^R \leq \begin{cases} \eta_v, & \kappa(s) = n + 1 \\ \text{vis}_{v,\kappa(s)}, & \kappa(s) \in C \\ 0, & \text{otherwise} \end{cases} \quad (34)$$

A selected trip must have exactly one launch truck and one recovery truck, and that truck must physically be at the launch/recovery node. Launch and recovery trucks are independent (flexible docking); with q indexed, trip q may recover to truck w_1 at k_1 while trip $q + 1$ launches from truck w_2 at i_2 . The robot analogues use g^L, g^R, z .

7.6 Multi-trip sequencing

For every $d \in D$ and $q \in \{1, \dots, Q_{\max} - 1\}$:

$$\text{used}_{d,q+1}^d \leq \text{used}_{d,q}^d \quad (35)$$

$$\text{lt}_{d,q+1}^d \geq \text{rt}_{d,q}^d + \tau_{\text{turn}} - M(2 - \text{used}_{d,q}^d - \text{used}_{d,q+1}^d) \quad (36)$$

and the robot analogues for $r \in R$. Trip $q+1$ may only launch after trip q has been recovered (plus turnaround), whenever both trips are flown. Per-trip launch/recovery times link to trucks and sorties in Section 7.10:

$$\text{lt}_{d,q}^d \geq a_{v,\iota(s)}^t + \text{dt}_{\iota(s),\sigma_1(s)} - M(1 - h_{v,d,q,s}^L), \quad \forall v, (d, q, s) \quad (37)$$

$$\text{rt}_{d,q}^d \geq a_{\sigma_m(s)}^d + \theta_{\sigma_m(s)} + \text{dt}_{\sigma_m(s),\kappa(s)} - M(1 - y_{d,q,s}), \quad \forall (d, q, s) \quad (38)$$

(35) canonically orders trips within a platform (eliminating trip-permutation symmetry). The per-customer arrival chain a_i^d is reused because each customer is served exactly once, so $a_{\sigma_m(s)}^d$ is well-defined regardless of which trip serves it.

7.7 Battery and en-route charging

For every $d \in D$ and $q \in \{1, \dots, Q_{\max} - 1\}$ (robot analogues for $r \in R$):

$$b_{d,1}^d = B_d \quad (39)$$

$$b_{d,q+1}^d = b_{d,q}^d - \sum_{s \in S^d} e(s) y_{d,q,s} + c_{d,q}^d \quad (40)$$

$$0 \leq b_{d,q}^d \leq B_d \quad (41)$$

$$c_{d,q}^d \leq r_d^c (\text{lt}_{d,q+1}^d - \text{rt}_{d,q}^d) \quad (42)$$

$$c_{d,q}^d \leq B_d \text{used}_{d,q+1}^d \quad (43)$$

(39)–(41) are the battery inventory (cf. VRP-DR Eqs. 30–33, 41–42); the lower bound enforces “cannot spend uncharged energy” through (40). (42) caps charged energy by the charging rate times the *aboard time* $\text{lt}_{d,q+1}^d - \text{rt}_{d,q}^d$ — the interval between recovery and next launch, during which the platform is by construction aboard the truck (cf. VRP-DR Eqs. 38–40; their “charging time $\leq$ truck leg time” is automatic here because aboard time is truck-synchronized). (43) forbids charging into the void. With $Q_{\max} = 1$, (39)–(43) are vacuous and (29) prices the single trip’s excess over the nameplate battery. *Modeling note:* under flexible docking, if trip $q+1$ launches from a different truck than trip q ’s recovery truck, the platform must transfer trucks mid-gap; charging should then accrue only on the truck-carried sub-intervals. The code requires same-truck consecutive trips unless a truck–truck rendezvous is modeled.

7.8 Truck timing with ready times (cyclic-ready)

$$a_{v,0}^t = 0, \quad v \in T \quad (44)$$

$$\text{dep}_{v,i}^t \geq a_{v,i}^t + \theta_i, \quad v \in T, i \in C \quad (45)$$

$$\text{dep}_{v,i}^t \geq a_{v,i}^t, \quad v \in T, i \in P \quad (46)$$

$$a_{v,j}^t \geq \text{dep}_{v,i}^t + \text{tt}_{ij} - M(1 - x_{v,i,j}), \quad v \in T, (i, j) \in A \quad (47)$$

$$a_{v,i}^t \geq e_i - M(1 - \text{vis}_{v,i}), \quad v \in T, i \in C \quad (48)$$

$$\ell_i^t \geq a_{v,i}^t - \bar{e}_i - M(1 - \text{vis}_{v,i}), \quad v \in T, i \in C \quad (49)$$

$\text{dep}_{v,i}^t$ is the truck’s *ready (departure) time* at node i : arrival plus service, plus any waiting for returning platforms. Separating arrival $a_{v,i}^t$ from availability $\text{dep}_{v,i}^t$ is what makes *cyclic* sorties feasible — with launch node = recovery node, the old form $a_{v,i}^t \geq a_{v,i}^t + \text{duration}$ was infeasible for any positive-duration sortie; now the truck simply waits, $\text{dep}_{v,i}^t \geq a_{v,i}^t + \text{duration}$. For acyclic sorties the behavior is unchanged (waiting becomes explicit rather than implicit).

7.9 Drone and robot time windows

$$a_0^d = 0, \quad a_0^r = 0 \quad (50)$$

$$a_i^d \geq e_i \sigma_i^d, \quad i \in C \quad (51)$$

$$a_i^r \geq e_i \sigma_i^r, \quad i \in C \quad (52)$$

$$\ell_i^d \geq a_i^d - \bar{e}_i, \quad i \in C \quad (53)$$

$$\ell_i^r \geq a_i^r - \bar{e}_i, \quad i \in C \quad (54)$$

$$a_i^d \leq \bar{e}_i + \ell_i^d + M(1 - \sigma_i^d), \quad i \in C \quad (55)$$

$$a_i^r \leq \bar{e}_i + \ell_i^r + M(1 - \sigma_i^r), \quad i \in C \quad (56)$$

Reporting note. For a customer i with $\sigma_i^d = 0$, (51) reduces to $a_i^d \geq 0$ and, since λ_W drives $\ell_i^d \rightarrow 0$, (55) reduces to $a_i^d \leq \bar{e}_i$. The variable a_i^d then has no effect on the objective, so the solver may set it to either bound of $[0, \bar{e}_i]$ with no physical meaning. The Arrival Times sheet flags such rows as `not_served_value_arbitrary` (same for a_i^r).

7.10 Ordered sortie timing

For $d \in D$, $q \in Q$, $s \in S^d$ with sequence $\sigma(s) = (\sigma_1, \dots, \sigma_m)$, launch node $\iota(s)$, recovery node $\kappa(s)$, and for every $v \in T$:

$$a_{\sigma_1}^d \geq a_{v,\iota(s)}^t + \text{dt}_{\iota(s),\sigma_1} - M(1 - h_{v,d,q,s}^L) \quad (57)$$

$$\text{dep}_{v,\kappa(s)}^t \geq a_{\sigma_m}^d + \theta_{\sigma_m} + \text{dt}_{\sigma_m,\kappa(s)} - M(1 - h_{v,d,q,s}^R) \quad (58)$$

and for every consecutive pair (σ_l, σ_{l+1}) in the sequence:

$$a_{\sigma_{l+1}}^d \geq a_{\sigma_l}^d + \theta_{\sigma_l} + \text{dt}_{\sigma_l,\sigma_{l+1}} - M(1 - y_{d,q,s}) \quad (59)$$

The robot analogues replace $a^d, h^L, h^R, y, \text{dt}$ with $a^r, g^L, g^R, z, \text{rt}$:

$$a_{\sigma_1}^r \geq a_{v,\iota(s)}^t + \text{rt}_{\iota(s),\sigma_1} - M(1 - g_{v,r,q,s}^L) \quad (60)$$

$$\text{dep}_{v,\kappa(s)}^t \geq a_{\sigma_m}^r + \theta_{\sigma_m} + \text{rt}_{\sigma_m,\kappa(s)} - M(1 - g_{v,r,q,s}^R) \quad (61)$$

$$a_{\sigma_{l+1}}^r \geq a_{\sigma_l}^r + \theta_{\sigma_l} + \text{rt}_{\sigma_l,\sigma_{l+1}} - M(1 - z_{r,q,s}) \quad (62)$$

Recovery constrains the truck's *ready time* $\text{dep}_{v,\kappa(s)}^t$, not its arrival: the recovery truck waits for the platform before departing. A late platform therefore propagates into that truck's $a_{v,n+1}^t$ and hence into ρ_v^t, ρ^* , and Γ .

7.11 Truck routing completeness

For every $v \in T$:

$$\sum_{j:(0,j) \in A} x_{v,0,j} = \eta_v \quad (63)$$

$$\sum_{i:(i,n+1) \in A} x_{v,i,n+1} = \eta_v \quad (64)$$

For every $v \in T$ and node $\in C$ (flow conservation):

$$\sum_{i:(i,\text{node}) \in A} x_{v,i,\text{node}} = \sum_{j:(\text{node},j) \in A} x_{v,\text{node},j} \quad (65)$$

For every $v \in T$, $i \in C$:

$$\text{vis}_{v,i} = \sum_{h:(h,i) \in A} x_{v,h,i} \quad (66)$$

For every $v \in T$:

$$\sum_{i \in C} \text{vis}_{v,i} \leq n \eta_v \quad (67)$$

$$\eta_v \leq \sum_{i \in C} \text{vis}_{v,i} \quad (68)$$

$$x_{v,i,j} \leq \eta_v, \quad (i, j) \in A \quad (69)$$

These constraints are not written explicitly in either friend formulation, which defines $x_{v,i,j}$ but not full route-continuity; they are the standard constraints needed for x to form valid physical truck routes.

7.12 Route-duration constraints

For every $v \in T$:

$$\rho_v^t \geq a_{v,n+1}^t - T_{\max} - M(1 - \eta_v) \quad (70)$$

$$a_{v,n+1}^t \leq T_{\max} + \rho_v^t + M(1 - \eta_v) \quad (71)$$

$$\rho_v^t \leq M \eta_v \quad (72)$$

$$\rho_v^t \leq \rho^* + M(1 - \eta_v) \quad (73)$$

Because every truck departs at time 0 ((44)), $a_{v,n+1}^t$ is that truck's route duration. ρ^* is forced, by (72)–(73) together with the objective minimizing $\lambda_T \rho^*$, to equal $\max_{v \in T, \eta_v=1} \rho_v^t$, i.e. the worst per-truck route-duration excess over $T_{\max}$.

Relation to drone/robot routes. The original PDF writes the route-duration condition for $m \in \{t, d, r\}$ (trucks, drones, robots separately). This implementation enforces it only through the truck end-depot arrival $a_{v,n+1}^t$. A drone/robot finishing a sortie late forces its recovery truck to wait (via (58)/(61)), which propagates into that truck's $a_{v,n+1}^t$ and hence into ρ_v^t , ρ^* , and Γ . So drone/robot lateness is captured indirectly through the truck term, not as a separate a_{n+1}^d, a_{n+1}^r penalty. See Section 10.

7.13 MTZ subtour elimination

For every $v \in T$:

$$u_{v,i} \leq n \text{vis}_{v,i}, \quad i \in C \quad (74)$$

$$u_{v,i} \geq \text{vis}_{v,i}, \quad i \in C \quad (75)$$

$$u_{v,i} - u_{v,j} + n x_{v,i,j} \leq n - 1, \quad i \neq j \in C, (i, j) \in A \quad (76)$$

7.14 Truck–platform pairing limits

For every $v \in T$:

$$\sum_{d \in D} \xi_{v,d} \leq \mu_d^{\text{trk}} \eta_v \quad (77)$$

$$\sum_{q \in Q} \sum_{s \in S^d} (h_{v,d,q,s}^L + h_{v,d,q,s}^R) \leq 2Q_{\max} \xi_{v,d}, \quad d \in D \quad (78)$$

$$\xi_{v,d} \leq \sum_{q \in Q} \sum_{s \in S^d} (h_{v,d,q,s}^L + h_{v,d,q,s}^R), \quad d \in D \quad (79)$$

$$\sum_{r \in R} \zeta_{v,r} \leq \mu_r^{\text{trk}} \eta_v \quad (80)$$

$$\sum_{q \in Q} \sum_{s \in S^r} (g_{v,r,q,s}^L + g_{v,r,q,s}^R) \leq 2Q_{\max} \zeta_{v,r}, \quad r \in R \quad (81)$$

$$\zeta_{v,r} \leq \sum_{q \in Q} \sum_{s \in S^r} (g_{v,r,q,s}^L + g_{v,r,q,s}^R), \quad r \in R \quad (82)$$

The upper-bound coefficient is $2Q_{\max}$, not 2: with up to $Q_{\max}$ trips per platform, a platform may launch and recover up to $Q_{\max}$ times each with one truck. $\xi_{v,d}$ ($\zeta_{v,r}$) still counts *distinct* platforms per truck, so (77) semantics are unchanged.

7.15 Launch/recovery node capacity limits

For every $v \in T$ and every node $w \in V$ that is a launch node of some $s \in S^d$:

$$\sum_{d \in D} \sum_{q \in Q} \sum_{\substack{s \in S^d \\ \iota(s)=w}} h_{v,d,q,s}^L \leq \begin{cases} \mu_d^{\text{dep}} \eta_v, & w = 0 \\ \mu_d^{\text{trk}} \text{vis}_{v,w}, & w \neq 0 \end{cases} \quad (83)$$

and for every recovery node $w \in V$ that is a recovery node of some $s \in S^d$:

$$\sum_{d \in D} \sum_{q \in Q} \sum_{\substack{s \in S^r \\ \kappa(s)=w}} h_{v,d,q,s}^R \leq \begin{cases} \mu_d^{\text{trk}} \eta_v, & w = n + 1 \\ \mu_d^{\text{trk}} \text{vis}_{v,w}, & w \in C \end{cases} \quad (84)$$

with the symmetric pair for robots using $g^L, g^R, \mu_r^{\text{dep}}, \mu_r^{\text{trk}}$. A platform may now launch twice from one node on different trips; the sums count trips correctly.

7.16 Symmetry breaking

$$\text{used}_{d,q+1}^d \leq \text{used}_{d,q}^d, \quad d \in D, q < Q_{\max} \quad (85)$$

$$\sum_{q \in Q} \sum_{s \in S^d} y_{d+1,q,s} \leq \sum_{q \in Q} \sum_{s \in S^d} y_{d,q,s}, \quad d \in \{1, \dots, |D| - 1\} \quad (86)$$

$$\sum_{q \in Q} \sum_{s \in S^r} z_{r+1,q,s} \leq \sum_{q \in Q} \sum_{s \in S^r} z_{r,q,s}, \quad r \in \{1, \dots, |R| - 1\} \quad (87)$$

(85) (same as (35)) canonically orders trips within a platform; (86) generalizes the old platform ordering to total trip counts. Valid: platforms are identical and trips are canonically ordered within each platform, so sorting platforms by trip count descending loses nothing.

7.17 Makespan

$$\Gamma \geq a_{v,n+1}^t - M(1 - \eta_v), \quad v \in T \quad (88)$$

$$\Gamma \geq \text{rt}_{d,q}^d - M(1 - \text{used}_{d,q}^d), \quad d \in D, q \in Q \quad (89)$$

$$\Gamma \geq \text{rt}_{r,q}^r - M(1 - \text{used}_{r,q}^r), \quad r \in R, q \in Q \quad (90)$$

$$\Gamma \geq 0 \quad (91)$$

Γ is the latest completion time over all used trucks and all flown platform trips. Unlike the previous model (makespan reported only), it is a decision variable entering the objective through (23).

8 Lossless Fleet Trimming

Before D and R are fixed, the raw fleet sizes from `parameters.json` (`NUM_DRONES`, `NUM_ROBOTS`) are trimmed to

$$|D| = \max(1, \min(\mu_d^{\text{trk}} \cdot |T|, n)) \quad (92)$$

$$|R| = \max(1, \min(\mu_r^{\text{trk}} \cdot |T|, n)) \quad (93)$$

The lossless argument is unchanged under multi-trip: (77) caps distinct usable drones at $\mu_d^{\text{trk}} \cdot |T|$, and (24) caps used platforms at n (each used platform serves at least one customer across its trips; each customer is served once), so any platform beyond $|D|$ (or $|R|$) is provably unused in every feasible solution. Multi-trip changes the *economics* (fewer platforms needed per customer), not the validity of the trim.

9 Time-Dependent Travel Times

The original PDF formulation assumes time-dependent travel times $t_{ij}(\tau)$. The MILP above is static: every timing constraint uses the free-flow matrix tt_{ij} . The codebase ships a time-dependent truck layer for a posteriori evaluation of solutions under $t_{ij}(\tau)$ and for time-dependent heuristic route evaluation (`scripts/time_dependent.py`, `scripts/add_td_matrices.py`).

Method. The working day $[0, H]$ is split into P equal periods. Each period p carries a speed factor $f_p \in (0, 1]$ relative to free-flow speed. The travel time for a departure at the start of period p is computed by *integrating speeds* — the classical construction of Ichoua, Gendreau & Semet (EJOR 144(3), 2003): find the smallest T with $\int_{\tau_0}^{\tau_0+T} f(\tau) d\tau = tt_{ij}$, where τ_0 is the departure time and $f(\cdot)$ is the stepwise speed profile (repeated cyclically past the horizon). Because speeds, not travel times, are the primitive, the FIFO property holds by construction: departing later can never arrive earlier. The generator asserts `verify_fifo` on every build.

Profiles. Speed factors are defined as resolution-independent piecewise segments over day fractions and discretized at period midpoints, so any P can be used:

- **two_peak:** two-peak urban day, maximum slowdown 2:1, within Figliozzi’s (TRE 48(3), 2012) 2.5:1 empirical bound;
- **mild:** small-city congestion (e.g. DeKalb, IL), slowest factor 0.80;
- **flat:** all factors 1.0, the static control.

Scope and files. Only the truck layer is time-dependent. Robot (pedestrian) congestion data does not exist publicly and drone airspace is uncongested, so dt_{ij} and rt_{ij} stay static; this is recorded in the instance provenance. For each profile the generator writes `truck_time_matrix_{profile}_p{p}.csv`, $p = 0, \dots, P-1$, with entry $[p][i, j] = tt_{ij}^p$ in hours, and merges provenance (period bounds, speed factors, citations, `td_fifo_verified`) into `parameters.json`. Flags: `--time-periods P`, `--td-profile`; existing instances are retrofitted with `scripts/add_td_matrices.py`.

Status in this model. The MILP of Sections 1–8 optimizes the static problem. The TD matrices are not decision-model inputs; they evaluate fixed routes under $t_{ij}(\tau)$ (e.g. re-timing a proven-optimal static route through peak periods) and will drive time-dependent heuristic search, with the exact static optimum as the baseline for optimality-gap measurement. Optimizing directly under time dependence would require a period-expanded MILP and is deliberately out of scope.

10 Deviations from the Friend Formulations

This model now *follows* the two friend formulations rather than deviating from them. Remaining differences:

- **Static optimization with time-dependent evaluation.** Both friend formulations assume $t_{ij}(\tau)$ throughout; this MILP optimizes static tt_{ij} , while Section 9 provides FIFO-safe time-dependent matrices for evaluating solutions and heuristics under $t_{ij}(\tau)$.
- **Route-duration penalty is truck-indexed only**, as discussed after (73), rather than summed separately over $m \in \{t, d, r\}$ as in the original PDF’s Eq. 8/Eq. 12. Drone/robot lateness still affects ρ^* and Γ indirectly through synchronization.
- **Energy replaces distance as the endurance primitive.** VRP-DR caps drone distance per sortie and models robot energy quadratically; here both platforms use the linear Wh/km energy $e(s)$ precomputed at generation, and the λ_E penalties price Wh of battery excess. For our hardware the drone battery is sized exactly to its range (energy binds); the robot battery is $\sim 5\times$ oversized, so robot charging is structurally minor at this scale.
- **Operational routing constraints** ((63)–(76)) are added because the friend formulations define $x_{v,i,j}$ but do not write full route-continuity constraints; without them x does not necessarily form a connected truck route from depot to depot.

- **Strict-model compatibility.** The previous strict model (hard capacity/endurance, cost-only objective) is recovered exactly with $\alpha = 1$, Γ dropped, $E_{\text{slack}} = 1$, and $\lambda_Q, \lambda_E^d, \lambda_E^r \rightarrow \infty$; the code’s `relaxed_objective` flag selects between them. Proven-optimal results under the strict model (slice6: 67.1552; slice8: 68.8452) do *not* transfer to the relaxed model, which has a strictly larger feasible region and must be re-solved for its own certificates.
- **Big-M, waiting penalty, and raw cost fields** (`big_M`, `waiting_penalty_weight`, `truck_variable_cost`, `drone_departure_cost`) from the raw TDRP-TW parameters are superseded: M is computed internally (Section 4), platform waiting is reported via the Sortie Timing Audit rather than penalized, and travel costs use $C_{veh}/C_{drone}/C_{rob}$ with the fixed-cost terms (13)–(15).

A full parameter-by-parameter audit is in `parameter_usage_audit.csv` and the README’s “Parameter Usage Audit” section.

A Side-by-Side Comparison with the Friend Formulations

✓ = aligned, ≈ = deviates (documented).

Aspect	Friend formulations	This implementation	Match
Sortie size	Original: one customer per sortie; VRP-DR: multi-visit	Multi-customer ordered sorties; <code>MAX_CUSTOMERS_PER_SORTIE</code> = 1 reproduces the original	✓
Multi-trip	VRP-DR: repeated dispatches per platform	Trip-indexed platforms (d, q) , $q \leq Q_{\max}$, with inter-trip sequencing	✓
Flexible docking	VRP-DR: independent launch/recovery trucks	$h^L \neq h^R$ ($g^L \neq g^R$) per trip; cross-truck trips allowed	✓
Cyclic/acyclic	VRP-DR: both	Self-loop legs (i, i) + truck ready times dep^t	✓
En-route charging	VRP-DR: battery + charging on truck	Per-sortie energy $e(s)$; battery $b_{p,q}$; charging $c_{p,q}$ during aboard time	✓
Travel times	Time-dependent $t_{ij}(\tau)$ throughout	Static tt_{ij} in the MILP; TD matrices for evaluation (Section 9)	≈
Customer service	Each customer served exactly once	Same	✓
Capacity	Original Eq. 12: constraints + λ_Q penalty	Excess ε_v^Q + λ_Q penalty (relaxed); per-sortie payload hard	✓
Endurance	Original Eq. 12: λ_E^d, λ_E^r penalties; VRP-DR: energy model	Battery excess $\varepsilon^{Ed}, \varepsilon^{Er}$ + penalties; linear Wh/km energy	✓
Synchronization	Sortie at nodes visited by <i>any</i> truck	Sortie paired to a specific truck, plus launch/recovery node capacities	≈ (stricter)
Time windows	Soft: penalty if late	Soft: lateness variables + λ_W penalty	✓
Route duration	Per mode $m \in \{t, d, r\}$: $a_{n+1}^{(m)} \leq T_{\max}$	Worst-truck excess ρ^* only; drone/robot lateness via synchronization	≈
Objective	Original Eq. 12: operating cost + 5 penalty terms; VRP-DR: $\alpha \cdot \text{cost} + (1 - \alpha) \cdot \text{makespan}$	$\alpha \cdot \text{OpCost} + (1 - \alpha) \Gamma + \text{Pen}_Q + \text{Pen}_E + \text{Pen}_T + \text{Pen}_W$	✓
Truck routing	$x_{v,i,j}$ defined; route continuity not written	Full flow conservation + MTZ subtour elimination	≈ (completed)
Robot network	Distinct pedestrian network $G^{(r)}$	Real OSM footpath/sidewalk network	✓
Cost model	$C_{ij}(\tau) = t_{ij}(\tau)C_w + d_{ij}C_{veh}$	Same, plus fixed activation costs f_t, f_d, f_r	✓